FEW-SHOT CALORIE-AWARE SNACK DETECTION

Sandy Zhang
Computer Science
Princeton University
Princeton, NJ 08544

Brian Zhou
Electrical and Computer Engineering
Princeton University
Princeton, NJ 08544

Jeremy Bao
Computer Science
Princeton University
Princeton, NJ 08544

December 18, 2025

`Github: https://github.com/jere-mybao/casi.`

1 Motivation and Goal

Have you ever eaten a random assortment of snacks and had no idea of how many calories you just consumed? While calorie tracking is often treated as a simple tracking task, estimating calories from images remains challenging. Food items can be irregularly shaped, visually similar, and partially occluded. The photos itself can introduce more ambiguity depending on the lighting, perspective, and depth. These challenges are especially true for snack foods, which are normally consumed casually without clear nutritional labels.

Recent advances in computer vision have enabled AI-based approaches to food recognition and calorie estimation, including object detection models such as YOLO and vision–language models (VLMs). While these models have shown strong performance, much of the work focuses on full meals or broad food categories, rather than individual snack items consumed by many college students.

In this work, we wonder if focusing on a narrow food domain with limited available data is sufficient for reliable calorie estimation. The goals of our investigation are to:

- Build a YOLO-style architecture from scratch to better understand the design of current detection models.
- Test whether a standard YOLO model trained in a few-shot setting can adapt across domains to accurately detect individual snack items.
- Compare the performance of state-of-the-art YOLO models with VLMs.

2 Background and Related Work

Most people typically calculate their calories by counting each piece of food they consume. However, more recent studies indicate that calorie counting is not as straight forward as it may seem [1]. And as interest in nutrition tracking has grown, AI has become a widely-adopted tool used for automating food analysis. Shonkoff et al. identify 52 published papers that used AI to estimate calorie and nutrient intake and found that relative errors for calorie estimation suggest that AI methods might have potential to surpass the accuracy of human estimation [2]. One AI-based approach for calorie estimation is leveraging computer vision, particularly object detection models such as YOLO to identify and localize food items in images.

YOLO Models There are several studies built on YOLO's algorithm to estimate calorie and food nutrition [3, 4]. P. et al. train YOLOv4 on the Food-101 and Food Nutrition Analysis + EDA datasets to identify nutrient breakup [3]. Their system was able to accurately recognize food items and match them with their nutritional information, potentially reducing the need for manual calorie tracking and lower errors in self-reported diets. Compared to CNN-based methods, YOLO is more efficient, achieving real-time performance at little to not cost of accuracy. This makes it a suitable choice for calorie estimation applications.

Vision-Language Models Vision–language models combine visual recognition with language reasoning for better interpretation of complex or ambiguous calorie estimation tasks. For the purpose of calorie estimation, a VLM can be used to (1) interpret photos where multiple items overlap, (2) handle foods that are not cleanly represented in the label set, and (3) give structured outputs that can be mapped to nutrition databases. Yao et al. test VLMs for calorie estimation and ingredient recognition using visual-textual alignment and lightweight fine-tuning to improve calorie tracking accuracy. They combine image features with textual descriptions of the food to achieve a robust understanding of portion size and dish composition [5] That said, VLMs rely on longer inference times that inhibit them from being useful for fast online tracking.

3 Approach and Implementation

3.1 YOLOv1 from Scratch

3.1.1 Problem Formulation

The YOLOv1 problem setting is single-stage object detection [6]. For a given input image $I \in \mathbb{R}^{H \times W \times 3}$, we want to predict a set of target labels $\mathcal{Y} = \{(c_k, b_k, p_k)\}_{k=1}^{N}$ where $c_k = \{1, ..., C\}$ is one-hot encoding of the class indices, $b_c = (x_k, y_k, w_k, h_k)$ is the normalized image coordinates centered at (x, y), and p_k is the probability that there is an object; the goal is to learn a function f_θ that takes an input image and outputs a set of ground truth detections.

YOLOv1 partitions an input image into a $S \times S$ grid. Each grid cell i provides a prediction of predicted one-hot encoded class vector $\hat{c}_i$, the B candidate bounding boxes $\hat{b}_{i,j} = (\hat{x}_{i,j}, \hat{y}_{i,j}, \hat{w}_{i,j}, \hat{h}_{i,j}), j = 1, ..., B$, as well as the associated confidence scores $\hat{p}_{i,j} \in [0, 1]$. The model output a tensor of shape $S \times S \times (B \times 5 + C)$.

We learn the parameters θ by minimizing a loss function that is defined by a weighted sum of the localization error of the bounding boxes, the confidence estimates of representative and non-representative bounding boxes, and the classification error of cells containing objects defined as the following:

$$
\begin{aligned}
\mathcal{L} = \lambda_{\text{coord}} \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbb{1}_{i,j}^{\text{resp}} \Big[& (x_i - \hat{x}_{i,j})^2 + (y_i - \hat{y}_{i,j})^2 \\
& + (\sqrt{w_i} - \sqrt{\hat{w}_{i,j}})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_{i,j}})^2 \Big] \\
& + \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbb{1}_{i,j}^{\text{resp}} \left(p_i^{\text{obj}} - \hat{p}_{i,j}^{\text{obj}} \right)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=1}^{S^2} \sum_{j=1}^{B} \left(1 - \mathbb{1}_{i,j}^{\text{resp}} \right) \left(0 - \hat{p}_{i,j}^{\text{obj}} \right)^2 \\
& + \sum_{i=1}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c=1}^{C} \left(p_{i,c}^{\text{cls}} - \hat{p}_{i,c}^{\text{cls}} \right)^2 .
\end{aligned}
\tag{1}
$$

with the indicator functions defined by:

$$
\mathbb{1}_{i,j}^{\text{resp}} = \begin{cases} 1, & \text{if box } j \text{ in cell } i \text{ is responsible for a ground-truth object,} \\ 0, & \text{otherwise.} \end{cases} \tag{2}
$$

$$
\mathbb{1}_{i}^{\text{obj}} = \begin{cases} 1, & \text{if cell } i \text{ contains the center of a ground-truth object,} \\ 0, & \text{otherwise.} \end{cases} \tag{3}
$$

In this equation, $\sqrt{w}, \sqrt{h}$ are used to prevent larger boxes from dominating the loss function and p_i^{obj} is defined as the IoU between the prediction and ground truth bounding boxes. The hyperparameters $\lambda_{coord} = 5$ and $\lambda_{noobj} = 0.5$ are used to balance localization errors and confidence prediction errors.

3.1.2 Architecture

We implemented the YOLOv1 architecture on the `PASCAL-VOC` dataset [7] just like the original paper [6]. We obtained the `PASCAL-VOC-2007` and `PASCAL-VOC-2012` datasets from Kaggle-hub and used it to pre-process the data into suitable image & label formats for downstream training. We could not find the pretrained Darknet backbone weights on PyTorch, so we used PyTorch's ResNet50 ImageNet checkpoints (`IMAGENET1K-V2`) as our backbone. Because of this, we had to make some modifications so that the detection head and fully-connected layers received the right tensor dimensions. For using YOLOv1 on `PASCAL-VOC`, we used the standard $S = 3, B = 2, C = 20$. As shown below, the original YOLOv1 architecture is a CNN with 24 convolutional layers followed by 2 fully connected layers. The convolutional layers extract features from the input image, and the fully connected layers outputs a $7 \times 7 \times 30$ prediction tensor.

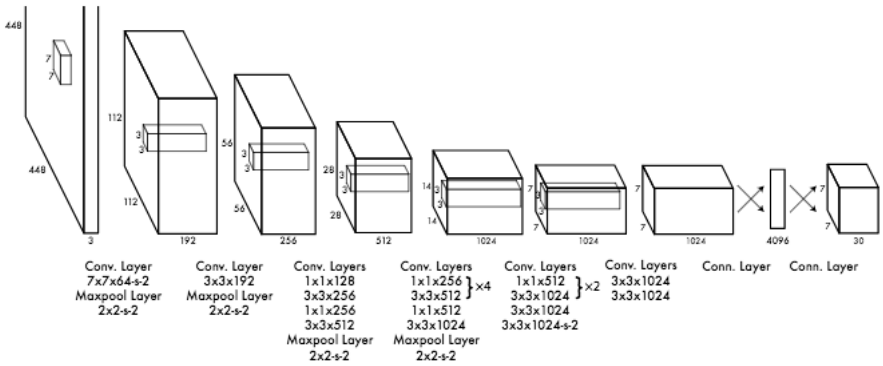


Figure 1: YOLOv1 model architecture [6]

3.1.3 Training

We train YOLO-v1 for 140 epochs with the original learning rate schedule using one node of the CS-operated `neuronic` cluster consisting of 8 NVIDIA L40 GPUs. We parallelized the model to increase training efficiency and also scaled up the number of workers accordingly. The wall clock time for training was just around 17 hours. The final checkpoints weights can be found on our Hugging Face. During training, we used WandB to log mean average precision (mAP) and train/test loss.

3.2 Calorie Estimation

3.2.1 Data Collection

We created our own dataset to not only manage the dataset size but also to ensure it reflected the snacks we eat regularly.

We gather images of candies (Swedish fish, Sour Patch Kids), cookies (Oreo, Milano cookies), and other common snacks (popcorn, roasted chestnuts, and pretzels). The photos were taken on a cutting board background with varying lighting conditions, sizes, quantities, and orientations.

We then used Human Signal, a cloud-hosted server that runs Label Studio, an open source labeling software to label our data. We manually annotate each image using YOLO-format bounding boxes with the goal of capturing object boundaries as accurately as possible. Our final dataset was around 200 images with roughly 3-5 bounding boxes per image.

Manual labeling gave us insight to the time-intensive process to gather a sizable dataset for labeled models, which visual features actually mattered, ambiguous situations, and how labeling inconsistencies affect model performance. We had to regather data on two occasions as model performance was severely impacted due to lack of data quantity and low probability scenarios in the validation dataset.

3.2.2 Data Augmentation

After the first iteration of training, we found that there were not enough instances of each item in the validation data, so we performed data augmentation on the dataset. We used the Albumentations library for augmentation that randomly

applied geometric and appearance changes and keeps the YOLO-format bounding boxes consistent. With regard to geometry, we used horizontal and vertical flips with a 50% probability and a 90% probability of performing small geometric shifts like translation (up to 6.25% of image size), scaling (up to 15%) and rotations up to 180 degrees to get different camera angles and orientations of objects. We introduced lighting and color variation on the pixel level as well: RandomBrightnessContrast (p = 0.8), color saturation shifts (p = 0.7), Gaussian noise (p = 0.5), and small blur (p = 0.3). After these two transformations, we also apply either splatter (dust artifacts) or contrast enhancement with a 40% chance. We transform boxes with the images using the bbox_params in the YOLO format, and we filtered boxes with very low visibility requiring 10% of the box to remain visible after the changes.

3.2.3 Model Training

The YOLOv12 model was trained on 488 training images and evaluated on 61 validation images. The images had 7 snack classes (Milano cookie, oreo, popcorn, pretzel crisp, roasted chestnut, Sour Patch kid, and Swedish Fish).

We fine-tuned from the pretrained YOLOv12 model weight rather than from scratch due to our small dataset to converge more efficiently. Training used a fixed input resolution that is standard for YOLO (640x640) and the traditional YOLO losses. To improve robustness, we performed data augmentation as said above. We controlled how we split the train/val/test to reduce leakage and ensure fair generalized metrics rather than memorization.

3.2.4 Hyper-parameter optimization

We perform hyperparameter optimization on learning rate and batch size on our training data over 25 epochs to select the best for the full 50 epoch training. We used Moraes et al. study on optimization for our other parameters due to compute constraints [8].

Table 1: Effect of learning rate and batch size on detection performance

Learning Rate (lr_0)	Batch Size	mAP@0.5–0.9 (25 epochs)	Best Epoch
0.001	8	0.60	25
0.001	16	0.58	23
0.003	16	0.57	24
0.003	8	0.56	25
0.010	16	0.53	24
0.010	8	0.51	25

Based on our results, we used a batch size of 8 and learning rate of 0.001 for our final training.

3.3 VLM

For the VLM, we use Google's Gemini 2.5 Flash as a baseline for calorie estimation from food images. The images used came from the test set and each image was evaluated on two different prompts: one without the look-up table that we used for our YOLOv12 training and one with the look-up table.

Prompt 1 (without the look-up table):

```
Estimate the TOTAL calories for the entire serving shown in the image.
Guidelines:
  • Identify each visible food item and estimate calories for each.
  • Sum to estimated_calories.
  • Provide calorie_range [low, high] capturing uncertainty (portion size,
    oils, sauces).
  • If it's not food or you can't tell, return low confidence and
    conservative estimates.
Allowed food categories:
  • milano cookie
  • oreo
  • popcorn
```

```
  • pretzel crisp
  • roasted chestnut
  • sour patch kid
  • swedish fish
```

Prompt 2 (with the look-up table):

```
Estimate the TOTAL calories for the entire serving shown in the image.
Guidelines:

  • Identify each visible food item and estimate calories for each and
    remember calories is per item and not portion size.
  • Sum to estimated_calories.
  • Provide calorie_range [low, high] capturing uncertainty (portion size,
    oils, sauces).
  • If it's not food or you can't tell, return low confidence and
    conservative estimates.

The only possible categories for the foods are as follows:

  • milano cookie
  • oreo
  • popcorn
  • pretzel crisp
  • roasted chestnut
  • sour patch kid
  • swedish fish

The calories are:


{
  "milano cookie": 65,      # per cookie
  "oreo": 70,               # per cookie
  "popcorn": 2.25,          # per one popcorn
  "pretzel crisp": 10,      # per crisp
  "roasted chestnut": 20,   # per chestnut
  "sour patch kid": 9,      # per candy
  "swedish fish": 22        # per candy
}
```

The results were then stored inside a CSV file.

4 Results and Discussion

4.1 Preliminaries from YOLOv1

Despite it's relatively simplistic architecture, YOLOv1 still achieves notable performance results at the end of training. Evaluation on a `PASCAL-VOC` test set of 4,952 images with 20 different class labels yields a mean average precision (mAP) of around 48%. It was interesting to see how the step-function-style learning rate schedule did not produce abrupt or distinctly observable changes in the loss or evaluation metrics, suggesting stable convergence.

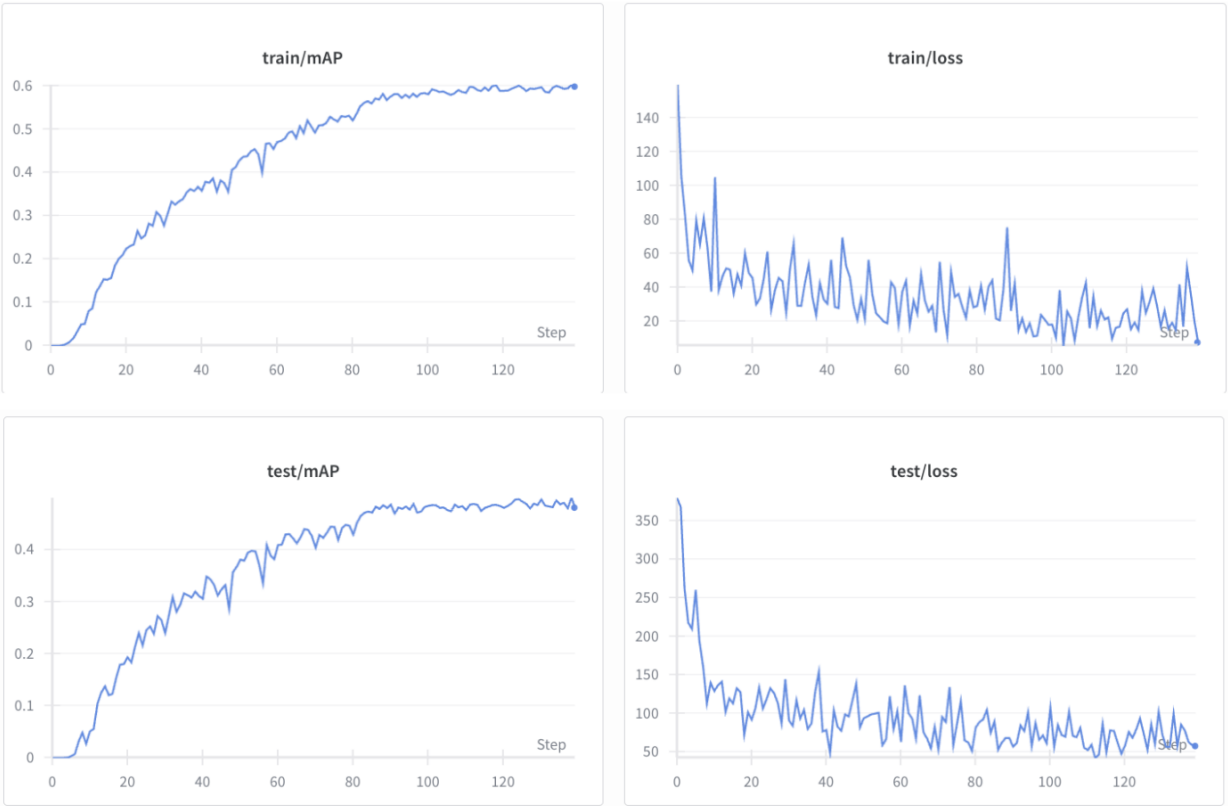


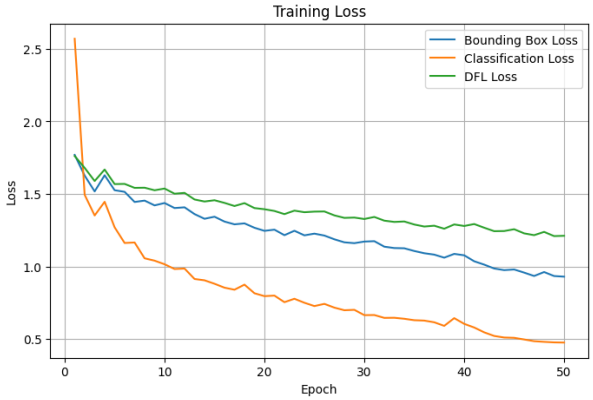
Figure 2: Training metrics for YOLOv1

Even though YOLOv1 achieves reasonable detection abilities on `PASCAL-VOC`, it would fall short in our calorie estimation task. It only predicts two bounding boxes per cell and is only able to finalize on one responsible object per grid cell. This architectural constraint is a big limitation in our use case because individual snacks are often cluttered together or overlapping. In particular, YOLOv2 addresses YOLOv1's limitations by introducing the idea of anchor boxes to predict mutiple final classes in a given spatial cell [9]. More recent models, such as YOLOv12, introduce architectural improvements that enhance feature extraction, efficiency, and robustness. For example, YOLOv12 incorporates an optimized backbone (R-ELAN), 7×7 separable convolutions, and FlashAttention-driven area-based attention [10, 11].

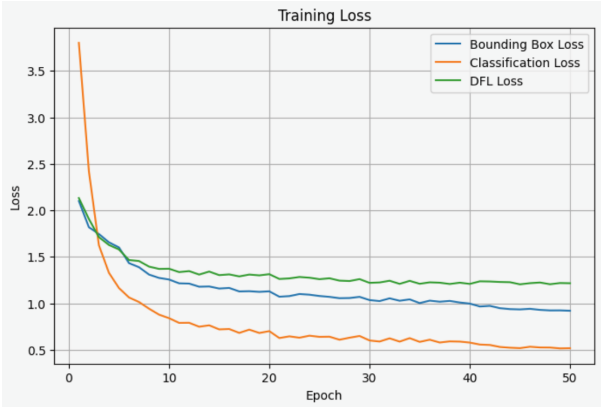
4.2 Experiments on YOLOv12

For the testing of YOLOv12, we trained using YOLOv12's standard regularization and with our own hyper-parameter optimization. During training using Ultralytics's automatic hyperparameter optimization, we observed the following: classification loss dropped from about 2.57 to about 0.4762, bounding box loss dropped from about 1.769 to about 0.9303, distribution focal loss dropped from about 1.762 to about 1.212. And during training using our hyper-parameters, we observed the following: classification loss dropped from about 3.801 to about 0.5166, bounding box loss dropped from about 2.100 to about 0.9195, distribution focal loss dropped from about 2.132 to about 1.215.

The classification loss measures whether the model is able to correctly predict the item category inside the detected bounding box [12]. The bounding box regression loss measures how well the predicted bounding box matches the ground-truth box, which is based on the complete intersection over union [13]. Distribution Focal Loss improves bounding box regression. It does so by predicting a probability distribution for box boundaries instead of a single value [14]. Figures 2 and 3 displays the drop in training loss over the epochs.



(a) Figure 2: YOLOv12 Training Loss with the standard optimization



(b) Figure 2: YOLOv12 Training Loss with our optimization

Figure 3: Training loss curves for YOLOv12

The two figures above compare the built-in Ultralytics auto-optimized (Figure 2) hyper-parameters to our own tuned parameters (Figure 3) based on the model's training loss over 50 epochs. In both instances, all losses decrease significantly in the first few epochs before leveling off. We expect this for a stable training run. Our run shows a sharper decrease early on and ends with slightly lower box loss. Classification and DFL loss converge to similar values. This trend suggests that our model is better able to localize (tighter bounding boxes).

As stated above, our optimization was intentionally lightweight due to computational constraints. We only tuned two knobs (learning rate and batch size) over a small grid of possibilities. For the rest of the parameters, we used recommendations from Moraes et al., using SGD rather than relying on the auto-optimizer's typical Adam-style optimizer[8]. Despite this limited scope, we still saw measurable improvements in training behavior, and we believe that a broader search that explores more parameters and ranges of values could further improve performance in future work.

The results displayed in Tables 2 and 3 illustrate the performance by object category.

Table 2: Detection performance by class using YOLOv12 optimization

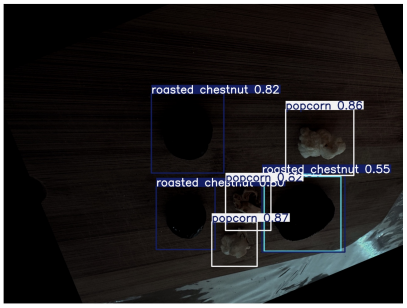
Class	Images	Instances	Precision	Recall	mAP50	mAP50:95
All	61	313	0.902	0.803	0.813	0.598
Milano Cookie	21	24	0.635	0.583	0.475	0.330
Oreo	24	24	0.950	0.790	0.892	0.725
Popcorn	38	85	1.000	0.862	0.905	0.630
Pretzel Crisp	39	57	0.964	0.895	0.913	0.729
Roasted Chestnut	47	57	0.986	0.930	0.959	0.706
Sour Patch Kid	20	34	0.967	0.872	0.858	0.604
Swedish Fish	29	32	0.815	0.686	0.689	0.461

Table 3: Detection performance by class using our optimization

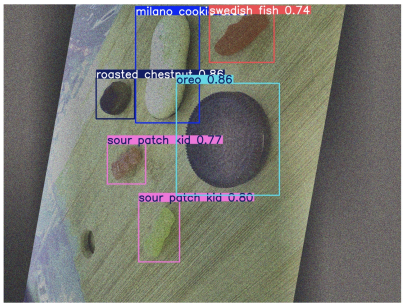
Class	Images	Instances	Precision	Recall	mAP@50	mAP@50:95
All	61	313	0.900	0.813	0.817	0.607
Milano cookie	21	24	0.688	0.583	0.496	0.380
Oreo	24	24	0.950	0.800	0.861	0.712
Popcorn	38	85	0.928	0.859	0.879	0.596
Pretzel crisp	39	57	0.962	0.885	0.929	0.728
Roasted chestnut	47	57	1.000	0.932	0.949	0.688
Sour Patch Kid	20	34	0.928	0.882	0.881	0.666
Swedish Fish	29	32	0.842	0.750	0.726	0.477

Tables 2 and 3 show the per-class performances of the trained models based on hyperparameter optimization. Overall, our optimization parameters outperform the auto-optimized baseline in most categories, including recall and both mAP while precision remains relatively consistent. This precision-recall tradeoff is one that we have seen consistently, in class, but the larger increase in recall compared to the relatively low decrease in precision is a win.

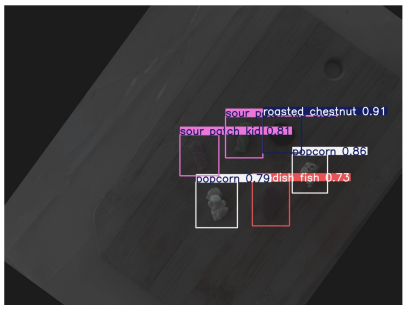
When looking at the individual classes, we see that performance improves significantly for Milano cookies, Sour Patch, and Swedish Fish while decreasing slightly for popcorn and Oreos. For Oreos, this may be because it had one of the smaller instance counts and also looks fairly similar to chestnuts in different scales and lighting environments.



(a) Image 1 Bounding Box



(b) Image 2 Bounding Box



(c) Image 3 Bounding Box

```
Ground truth calories:
  popcorn: 3 × 2.25 = 6.75 cal
  roasted chestnut: 2 × 20 = 40 cal
  oreo: 1 × 70 = 70 cal
GT total: 116.75 cal
Predicted calories:
  popcorn: 3 × 2.25 = 6.75 cal
  roasted chestnut: 3 × 20 = 60 cal
  oreo: 1 × 70 = 70 cal
Pred total: 136.75 cal
```

(d) Image 1 Calorie Count

```
Ground truth calories:
  milano cookie: 1 × 65 = 65 cal
  oreo: 1 × 70 = 70 cal
  swedish fish: 1 × 22 = 22 cal
  sour patch kid: 2 × 9 = 18 cal
  roasted chestnut: 1 × 20 = 20 cal
GT total: 195 cal
Predicted calories:
  oreo: 1 × 70 = 70 cal
  roasted chestnut: 1 × 20 = 20 cal
  sour patch kid: 2 × 9 = 18 cal
  swedish fish: 1 × 22 = 22 cal
  milano cookie: 1 × 65 = 65 cal
Pred total: 195 cal
```

(e) Image 2 Calorie Count

```
Ground truth calories:
  sour patch kid: 2 × 9 = 18 cal
  roasted chestnut: 1 × 20 = 20 cal
  popcorn: 2 × 2.25 = 4.5 cal
  swedish fish: 1 × 22 = 22 cal
GT total: 64.5 cal
Predicted calories:
  roasted chestnut: 1 × 20 = 20 cal
  popcorn: 2 × 2.25 = 4.5 cal
  sour patch kid: 2 × 9 = 18 cal
  swedish fish: 1 × 22 = 22 cal
Pred total: 64.5 cal
```

(f) Image 3 Calorie Count

Figure 4: Bounding Box and Calorie Count for 3 Test Set Images

The groups of images above show three examples from our test set to evaluate YOLO performance. The model was trained with our own hyperparameter optimization. We can see that in Figures (3b) and (3c), the model is successfully able to find all the snacks in the image and place accurate bounding boxes. Based on the bounding boxes, it calculates calories from a look-up table, and we see that the calorie count matches the ground truth. Moreover, these images in the test set have been subject to image augmentation, so we can see that the model is fairly robust despite strangely colored Oreos and lighting changes.

In Figure (3a), we see that the model is close to accurate, but classifies an additional chestnut (albeit with low confidence at 0.55). This is likely due to shadow caused by the change of light and the trend that Oreos and chestnuts have similar colors in the dark, so the only differentiator is the size, which can change depending on orientation and scaling. A way to solve for this could be to increase the confidence interval in which we draw a bounding box or perform additional data augmentation to increase model reliance on more robust features.

4.3 Experiments on VLM

To test with the VLM we used our optimized YOLOv12 model to compare the estimated calories. We compared our YOLOv12 model, the VLM without the look up table, and the VLM with the look up table to our ground truths. All

evaluations were done on the test set images. This ensures that the performance we observe reflects generalization to unseen data.

Table 4: Performance Compared to Ground Truth

Model	MAE	RMSE	MAPE (%)
VLM with Lookup	15.96	35.52	18.21
VLM without Lookup	43.98	67.90	48.75
YOLOv12	7.80	21.83	7.50

Figure 5: Scatter plots of predicted vs ground truth

From Figure 4 and Table 4 we see that YOLOv12 actually does better than the VLM in both cases. It achieves the lowest Absolute Error (MAE) of 7.80, Root Mean Squared Error (RSME) of 21.83 and Mean Absolute Percentage Error (MAPE) of 7.50. VLM without the lookup table performed worse than just a horizontal line, indicated by a negative $R^2(-0.856)$, likely because without specific boundaries, the VLM has difficultly narrowing down to the specific dataset. We see that given the lookup table, the VLM performs much better, but still lags behind significantly compares to the YOLOv12 performance. Both VLM with Lookup and YOLO models tend to underestimate the total calories while VLM without lookup overestimates. We see an overall strong correlation for the YOLOv12, but significant outliers still exist. This could be due to the darkness of some photos after data augmentation that makes it extremely hard to differentiate the actual item from an almost pitch black background. Overall, these results indicate that directly detecting each food item and using a look-up table of calorie values can achieve a more accurate calorie estimate than relying on language-based reasoning.

4.4 Strengths and Weaknesses of YOLOv12 and VLM

Our YOLOv12 pipeline estimates calories by detecting each snack instance and converting those counts into calories with a fixed table. This structure makes it easy to audit and understand the final result. The VLM is asked to directly infer total calories from the image either with the lookup table or without. On the test set, YOLOv12 achieves significantly lower error and has the highest R^2 score.

YOLOv12 strengths. YOLOv12 splits the representation into multiple stages. This split helps us focus on the part of the model that is the weakest, and this system improves audibility especially given our narrow range of identification. The structure allows for granular debugging and targeted data collection since we can see which scenes or classes drive the most error and create more examples around those cases. The detector also very naturally supports per-class performance which can help us account for bias or drift in a specific class and address the matter accordingly. YOLOv12 also yields a computationally efficient inference so we can have more real-time tracking and results.

YOLOv12 weaknesses. The biggest limitation of the YOLOv12 model is the rigidness of the model to a distribution shift or visually similar objects. If the detection misses an object or classifies a false positive, the final calorie count is necessarily higher or lower respectively since the calorie computation is hard count-based. This phenomenon is exacerbated by occlusion and clutter. Moreover, the current approach assumes that each detected instance corresponds to a standard unit. While this may be true for smaller snacks like a Swedish Fish, it may not hold for a pretzel that is broken in half.

VLM strengths. The VLM is more flexible since it combines contextual cues with cases where the image is different from the training distribution. It also produces a natural language justification, which makes its results more interpretable.

As we see, the VLM benefits greatly from explicit constraints. After being given the per-item lookup table, performance improved drastically from and MAE of 43.98 to 15.96. This improvement suggets that the primary failure is not necessarily the vision recognition, but instead calibration. Without guidelines, the VLM falls back to generic heuristics which may not match the dataset. We could have also used a stronger model or did some optimization on the prompt given to achieve better results.

VLM weaknesses. Despite giving the VLM the exact calorie counts that the YOLO model was given, it still outperformed YOLOv12 on all three error metrics. A likely reason for this gap is because the VLM must implicitly detect, count, and map to calories in one step (similar to what YOLO was meant to do with object segmentation and detection!). Thus, small errors in counting or uncertainty can result in larger deviations. Moreover, VLMs typically give more generalized estimations like portion sizes rather than individual pieces of snacks (a bag of chips vs. individual chips).

These results show that there is an opportunity for a hybrid model that uses YOLOv12 to produce candidate regions and counts and then uses a VLM to verify ambiguous regions.

5 Conclusions and Future Work

Our project evaluates whether reliable calorie estimation is achievable in a few-shot setting with a small domain of snacks using models that explicitly separate recognition from calorie computation. We used a small manually-labeled dataset with data augmentation to train YOLOv12 to detect seven snack calories and use a fixed per-item lookup table to compute calories. On the test set, our model achieved strong overall performance and correlation with the ground truth. We compared YOLOv12 against a Gemini 2.5 Flash powered VLM model. For calorie estimation, YOLOv12 significantly outperformed both the contrained and unconstrained VLM. These findings support that when we have a clearly defined per-item calorie system, a detection and count pipeline can be more accurate than a general large language model.

There are several future directions this project can take. One direction is to expand the dataset. Ideally, we would be able to detect a larger domain of snacks that are commonly eaten across all college students and not just our small group. This would mean more labeled data, and we could also experiment with models with automatically bound-boxes and labeling to greatly augment our data without the exponential increase in human hours. Another direction is estimating portion size. We currently assume all individual snacks have the same calorie count, but this is not necessarily true for a pretzel that is broken in half! While this assumption is more likely to hold for smaller candies that don't deform as easily, we could train the model to identify the percentage of a snack that is there for greater accuracy in calorie estimation. Moreover, we could create a pipeline that combines the YOLOv12 model with a VLM where the YOLO model segments the images and the VLM confirms and quantifies the percentage of a snack present within the specific bounding box to leverage the strengths of both models. A final direction could be increasing the model's robustness to clutter. Our current datasets mostly separate each individual snack to make detection easier, but snack bowls are often mixed together and overlap, making it hard for capture certain snacks at the bottom of the bowl. It's unclear if the model is able to perform well in this scenario. We included photos with overlapping snacks but ideally we would include more data of snacks in cluttered environments.

6 Acknowledgments

We would like to thank our mentor Will Hwang for giving us feedback as we worked through our project. We would also like to thank Dr. Russakovsky and Dr. Ramaswamy for providing us with advice and teaching us the computer vision fundamentals necessary for this work.

References

[1] Gabija Didžiokaitė, Paula Saukko, and Christian Greiffenhagen. Doing calories: The practices of dieting using calorie counting app MyFitnessPal. In Btihaj Ajana, editor, *Metric Culture*, pages 137–155. Emerald Publishing Limited.

[2] Eleanor Shonkoff, Kelly Copeland Cara, Xuechen (Anna) Pei, Mei Chung, Shreyas Kamath, Karen Panetta, and Erin Hennessy. AI-based digital image dietary assessment methods compared to humans and ground truth: a systematic review. 55(2):2273497.

[3] Nivedhitha P, Anurithi P, Meenashree S S, and Pooja Kumari M B. FOOD NUTRITION AND CALORIES ANALYSIS USING YOLO. In *2022 1st International Conference on Computational Science and Technology (ICCST)*, pages 382–386. IEEE.

[4] Fitroh Romadhon, Faisal Rahutomo, Joko Hariyono, Sutrisno, Meiyanto Eko Sulistyo, Muhammad Hamka Ibrahim, and Subuh Pramono. Food image detection system and calorie content estimation using yolo to control calorie intake in the body. 465:02057.

[5] Dongyu Yao, Keling Yao, Junhong Zhou, and Yinghao Zhang. Caloraify: Calorie estimation with visual-text pairing and lora-driven visual language models, 2024.

[6] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection, 2016.

[7] Mark Everingham, S. M. Ali Eslami, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The pascal visual object classes challenge: A retrospective. 111(1):98–136.

[8] André Magalhães Moraes, Luiz Felipe Pugliese, Rafael Francisco dos Santos, Giovani Bernardes Vitor, Rodrigo Aparecido da Silva Braga, and Fernanda Rodrigues da Silva. Effectiveness of yolo architectures in tree detection: Impact of hyperparameter tuning and sgd, adam, and adamw optimizers. *Standards*, 5(1), 2025.

[9] Joseph Redmon and Ali Farhadi. Yolo9000: Better, faster, stronger. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[10] Mujadded Al Rabbani Alif and Muhammad Hussain. Yolov12: A breakdown of the key architectural features, 2025.

[11] Yunjie Tian, Qixiang Ye, and David Doermann. Yolov12: Attention-centric real-time object detectors, 2025.

[12] Muhammad Yaseen. What is yolov8: An in-depth exploration of the internal features of the next-generation object detector, 2024.

[13] Zhaohui Zheng, Ping Wang, Dongwei Ren, Wei Liu, Rongguang Ye, Qinghua Hu, and Wangmeng Zuo. Enhancing geometric factors in model learning and inference for object detection and instance segmentation. *IEEE transactions on cybernetics*, 52(8):8574–8586, 2021.

[14] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-iou loss: Faster and better learning for bounding box regression. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 12993–13000, 2020.